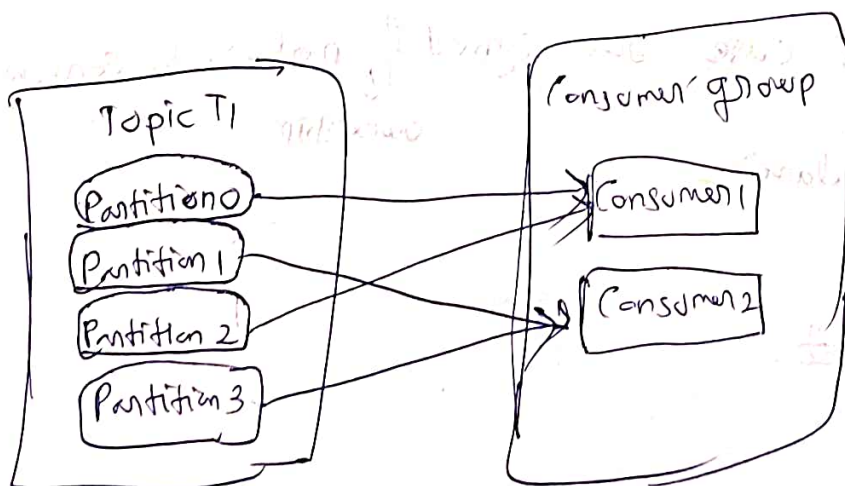
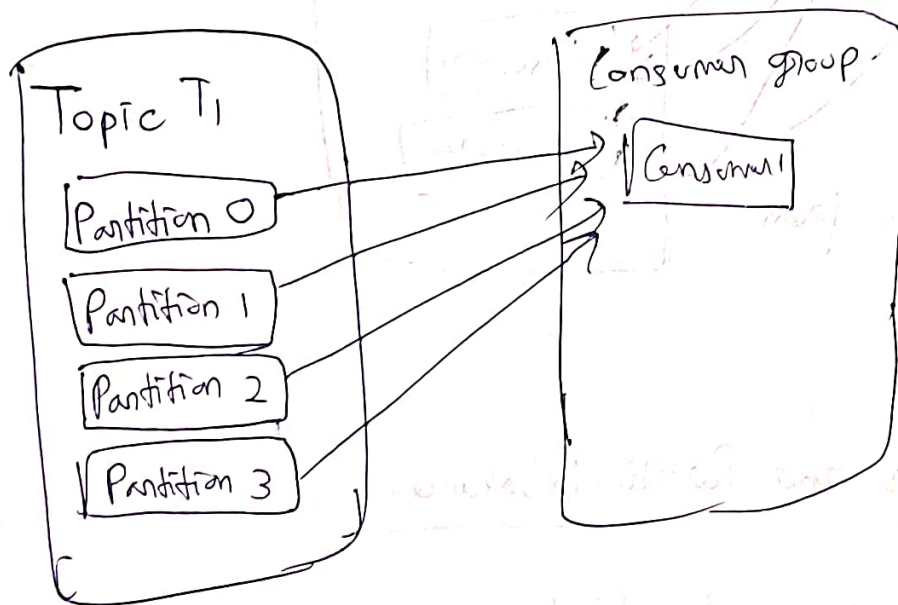
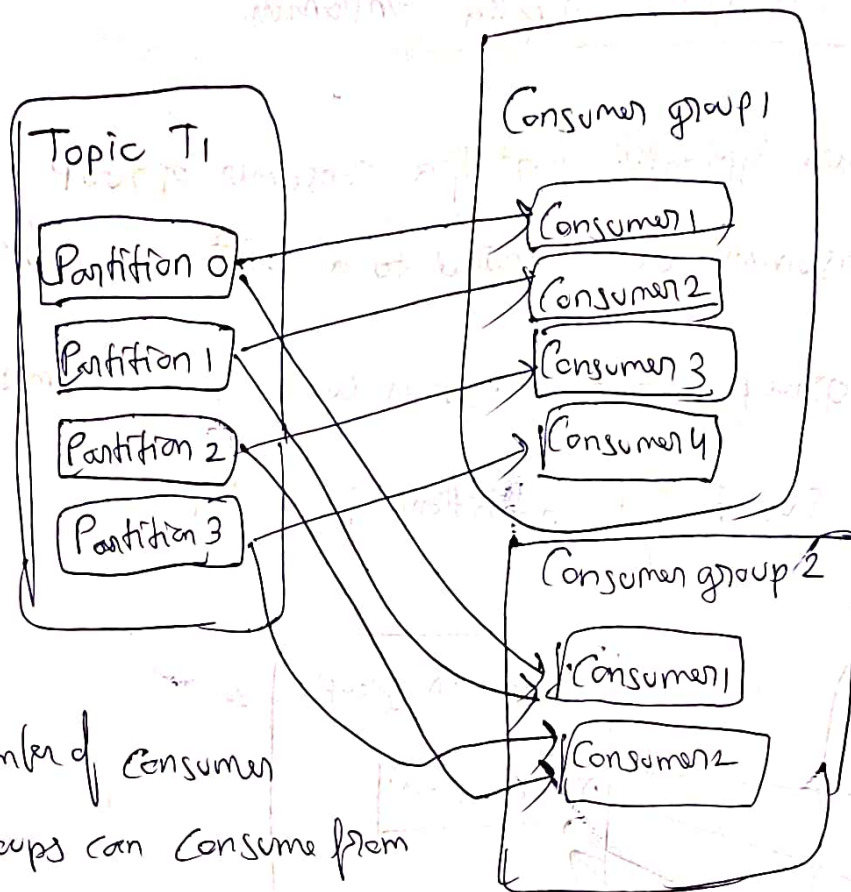


Chapter 4 Kafka Consumers.

Kafka consumers are typically part of a consumer group. When multiple consumers are subscribed to a topic and belong to same consumer group, each consumer will receive messages from a different subset of the partitions in a topic.



If no. of consumers $>$ no. of partitions then those extra consumers will sit idle.



Any number of consumer groups can consume from a topic.

Consumer Groups and Partition Rebalance

Let's say, new partition gets added or some consumers get added or removed in that case reassignment of partition to consumer happens ownership.

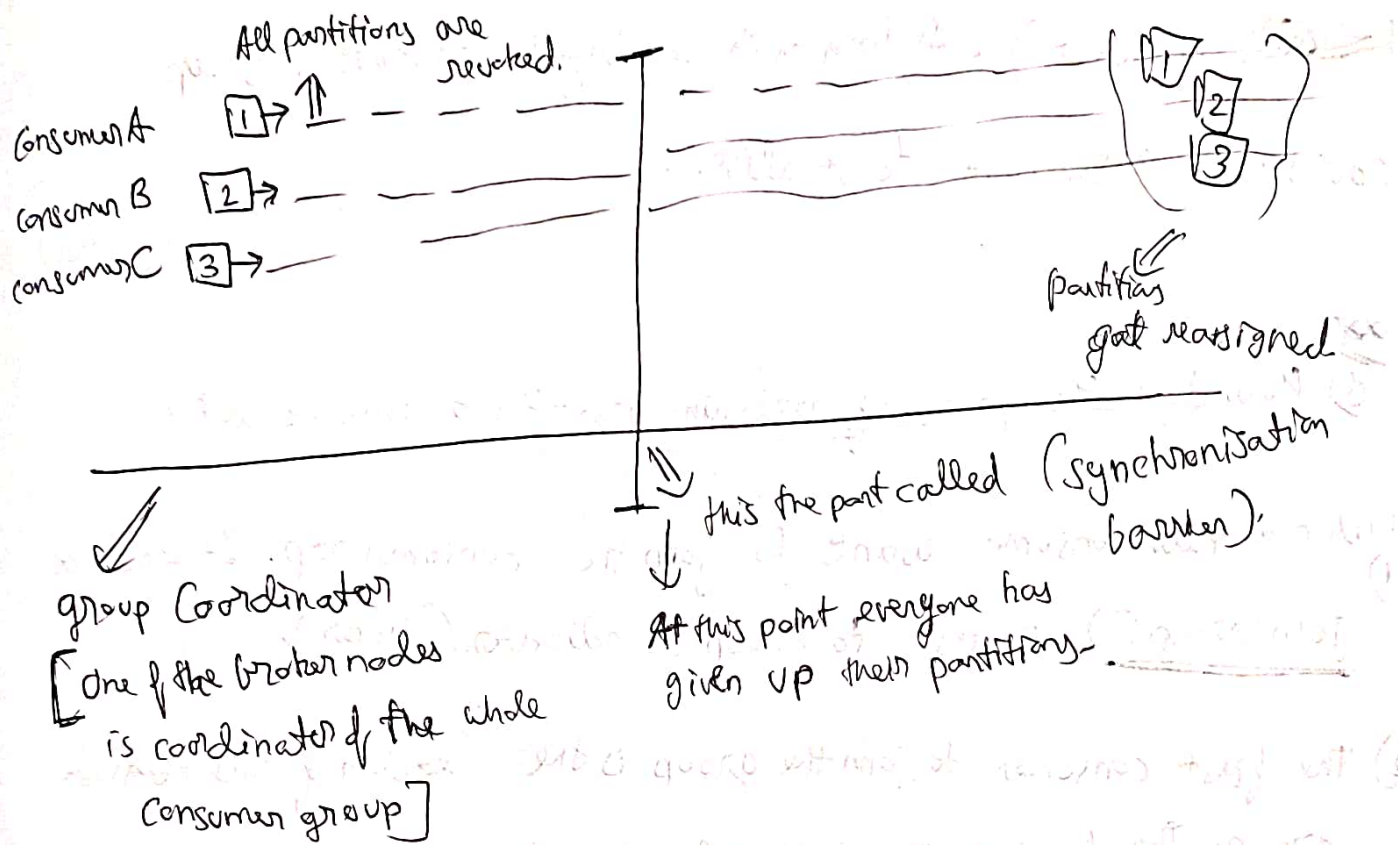
This is called rebalancing.

2 types of Rebalancing

P.T.O

Eager Rebalances. - During eager rebalances, all consumers stop consuming, give up their ownership of the partition, then all re-join the group with brand new assignments.

This is a short-window of unavailability of entire consumer group.



Cooperative rebalances → {Incremental rebalances}.

this happens in two phases

1st phase) The consumer group coordinator (a broker) informs the consumer group that a few partitions are going to be reassigned. Consumers stop consuming from those partitions.

2nd phase) Consumer group leader assigns new partitions to their new consumers. This takes a few iterations to stabilize but this does not mean complete stop the world (like GC or in eager rebalancing).

~~xxx~~ Group Coordinator

Consumers maintain a membership in consumer group and they have a group coordinator which is a Kafka broker and then consumers send periodic heartbeats (sent by background thread of consumer). As long as its sending heartbeats, group coordinator assumes it to be alive.

~~xxx~~

Q) How does the process of assigning partition to consumers work?

1) When a new consumer wants to join the consumer group. It sends a joinGroup() request to group coordinator. (Broker).

2) The first consumer to join the group is the leader of the consumer group. The leader receives a list of all consumers that sent a heartbeat and are part of the group.

3) ~~xxx~~ leader is responsible for assigning partitions to the ~~group~~ consumers in the group. They do this by writing an implementation of Interface Partition Assigner.

4) Leader sends this (partition-consumer) assignment to group coordinator and group coordinator sends this to all the consumers.

5) group coordinator and leader has all the list mappings no-one else.

Static group membership -

group.instance.id → This is basically given each consumer a static id.

On the group Coordinator [Broker] } ~~which is the~~

caches this consumer id → [partitions which were assigned to it]

Well now if the consumer restarts a rebalance will not have to be triggered as the result is cached in ~~the~~ group Coordinator.

So upon restart it picks up those partitions again.

Some how if ~~partitions~~ consumer is out beyond the session.timeout.ms then it is considered to be forgotten.

New thing I learnt) subscribing to Topics. using Regular Expression.

```
consumer.subscribe(Pattern.compile("test.*"));
```

↓

Subscribes to all topics having test as prefix

The Poll Loop

```
while (true) {  
    consumers.poll(timeout);  
    // do the processing  
}
```

long poll, it waits if content is not there

we will read next that if consumer has not polled for messages within a certain interval it is considered dead

1st time you do a `.poll()` call, then it is responsible to find the group coordinator. ~~Then it~~

~~Then~~ it will get the subset of partitions assigned to it

If consequent `.poll()` is not called within the interval of `(max-poll-interval-ms)` $\Rightarrow$ then that consumer is considered dead

Thread Safety $\rightarrow$

If you want to use one consumer and its application have multiple threads. You can make use of multithreading to make a consumer act as multiple consumers.

Few important configurations of Consumer.

`fetch.min.bytes` → minimum amount of data the consumer wants from the broker in one `poll()` call. If that amount of data is not there then wait.

Imagine if consumer CPU util is going high, so increase this parameter so less frequent polling will happen.

`fetch.max.wait.ms` → By default 500ms. Imagine if it never even fills up, ~~the~~ the `fetch.min.bytes`, then it might be `poll(timeout)` will happen but that's not the case, It waits till this time and returns and gets that amount of data.

`fetch.max.bytes` → This is a broker level config, Imagine influx of data is very fast. So it's the upper limit after which broker caps the record and sends to consumer.

So `fetch.max.bytes` → this is each broker level as it can be assigned to multiple partitions.

`max.poll.records` → No. of records not the size in MB of something.

`max.partition.fetch.bytes` → Having upper limit cap on how ~~many~~ much data will be sent from each partition.

`heartbeat.interval.ms` → Intervals at which consumer will send heartbeats to ~~gcp~~ coordinator.

`session.timeout.ms` → Longest the consumer can go without sending a heartbeat.

max-poll-interval-ms $\rightarrow$ max a consumer ~~could go~~ can go without polling before its considered dead.

Different from heartbeat

request-timeout-ms $\rightarrow$ max time consumer waits ~~to~~ for a response from broker before it closes tcp connection with consumer.

auto-offset-reset $\rightarrow$ lets say a consumer comes back after being dead, lets say group-coordinator is unable to find which offset it is supposed to read from again.

3 options

- 1) Latest $\rightarrow$ end of the partition
- 2) Earliest $\rightarrow$ Start of the partition
- 3) None $\rightarrow$ error.

enable.auto.commit $\rightarrow$ If set to true, application automatically commits offsets to broker.

if false $\rightarrow$ ~~cannot~~ write manual code to do that

Partition-assignment-strategy $\rightarrow$ We learnt that leader of the consumer group, is responsible for assigning partitions to leaders by implementing PartitionAssigner.

Few Strategies

Range $\rightarrow$  $\rightarrow$ assigning ranges to consumers.

RoundRobin $\rightarrow$ as name suggest round robin.

StickyAssigner $\rightarrow$ during rebalance eager rebalancing but tries to minimise change

Cooperative StickyAssigner $\rightarrow$ cooperative rebalancing but round robin ~~and~~.

offset-retention-minutes → Broker side config

If consumer group becomes inactive, broker starts countdown and broker stays inactive beyond this then it does not keep ~~partitions~~ offset for that consumer group anymore.

Commits & Offsets

Kafka has a special topic called `__consumer_offsets` topic where it pushes the last offset committed for each topic.

When a new consumer joins after a rebalance and is assigned a partition, the `auto.offset.reset` → (Earliest/latest) if latest is used, it will pick up the latest offset for that partition.

Automatic Commit → Easiest way to handle this is by using `(enable.auto.commit=true)` then maybe every 5 seconds the consumer will commit the last offset for the batch it received.

Note → Here it's not committing the processed messages. It's committing the last batch it received.

We all know this can lead to missing or processing a lot of messages. We can commit messages which we were not processing at all. Developers don't have enough control over processing duplicate messages.

Commit Current Offset →

enable-auto-commit → double this is not reliable,

go for enable-auto-commit = false

and make use of commitSync().

```
while (true) {
```

```
    poll() logic
```

```
}
```

```
try {
```

```
    consumers.commitSync();
```

```
}
```

```
catch (Exception e) {
```

```
}
```

problem is that its blocking ~~off~~
of course are ~~can do~~ retry.
But takes hit on performance.

Asynchronous Commit →

Since commitSync() is blocking hence commitAsync() can be used.
But the thing is since its non blocking so it does not retry

lets say an offset commit of offset → 2000 failed,

but at later point offset of 3000 already got committed,
So its fine only

As in producer, we had `async send()` ~~and~~ and a callback to handle errors,

similarly to handle offset commit errors we can handle, using callbacks.

```
consumer.commitAsync(new OffsetCommitCallback() {
```

```
@Override
```

```
public void onComplete(Exception e, ...) {
```

≡ do whatever you want

```
}
```

```
}
```

(Combination of Sync & Async Commits.)

```
while (!closing) {
```

```
poll()
```

```
commit commitAsync()
```

```
}
```

```
consumer.commitSync();
```

otherwise, async commit of offsets.

Not here, how only after consumer shutdown or before rebalance, to be extra safe we are

doing ~~commitAsync()~~
 `commitSync()`

Committing a specified offset

When we call `commitAsync()` or `commitSync()`, the thing is, it sends the last offset of the batch that we ~~processed~~ processed.

But now let's say I want to send offset for each record for better guarantee, then I would use another flavor of sending these offsets.

`map < TopicPartition, OffsetMetadata > mp = new HashMap<>();`

`mp.put(new TopicPartition(record.topic() record.topic(), record.partition() record.partition()),
new OffsetMetadata(record.offset() + 1, "-"));`

`commitAsync( mp );`

So next consumer reads from next message

We can do this

at intervals
also by

keeping a counter also

or maybe at time intervals

Rebalance Listener → When a consumer is about to go into rebalance state,

if might a that developer wants to do something before that like closing tcp/DB connections.

Some APIs are given by kafka to handle this.

onPartitionAssigned (partition) → whatever logic you write here will execute once the consumer gets the partition, but is yet to start consuming.

onPartitionsRevoked (partition) → this flow executes when consumer has to give up partitions and can do whatever just before disconnecting from the partition.

↓
(Cooperative/incremental rebalancing)

Consuming Records with Specific Offsets

In some scenarios we don't want earliest or latest offset consumption. Sometime we want something like maybe an offset or some specified offset.

In that case consumer.seek() method is used.

consumer.seek(topic, partition, offset);

We know there are

Custom serializers & deserializers

like in producer. for String, Avro etc

~~202~~

When we do Consumer.subscribe(topic_name);

→ leader assigns us partition.

Another flavour, we can use : consumer.assign(List<PartitionInfo>),

using this we can use consumers to assign partitions directly.